COMSATS University Islamabad

Attock Campus

Department Of Computer Science

Course	*IS*
Instructor	***Mam Ambreen***

- **Student Details**

Registration No.	*Name*
FA24-BSE-042	Sami Ullah

Question no. 1: Suggest 3 programing changes in above Diffie-Hellman Code, your updated code should generate code output.

```
def power(a, b, p):
    if b == 1:
        return a
    else:
        return pow(a, b) % p

# Main function
def main():
    # Both persons agree upon the public keys G and P
    # A prime number P is taken
    P = 23
    print("The value of P:", P)
    # A primitive root for P, G is taken
    G = 9
    print("The value of G:", G)
    # Alice chooses the private key a
    # a is the chosen private key
    a = 4
    print("The private key a for Alice:", a)
    # Gets the generated key
    x = power(G, a, P)
    # Bob chooses the private key b
    # b is the chosen private key
    b = 3
    print("The private key b for Bob:", b)
    # Gets the generated key
    y = power(G, b, P)
    # Generating the secret key after the exchange of keys
    ka = power(y, a, P) # Secret key for Alice
    kb = power(x, b, P) # Secret key for Bob
    print("Secret key for Alice is:", ka)
    print("Secret key for Bob is:", kb)

if __name__ == "__main__":
    main()
```

Code:

```
[Running] python -u "d:\Subjects\IS LAB\Lab 6\te
The value of P: 23
The value of G: 9
The private key a for Alice: 4
The private key b for Bob: 3
Secret key for Alice is: 9
Secret key for Bob is: 9
```

Question no. 2: Search about latest attack on key exchange algorithms and provide its implementation in python.

```
def modexp(base, exponent, modulus):
    return pow(base, exponent, modulus)

def dh_public_key(prime, generator, private_key):
    return modexp(generator, private_key, prime)

def dh_shared_secret(prime, received_public, private_key):
    return modexp(received_public, private_key, prime)

def simple_xor_encrypt(message, key_byte):
    return bytes([b ^ key_byte for b in message])

def demo_mitm_attack():
    prime = 23
    generator = 5

    alice_private = 6
    bob_private = 15
    attacker_private_a = 13
    attacker_private_b = 17

    alice_public = dh_public_key(prime, generator, alice_private)
    bob_public = dh_public_key(prime, generator, bob_private)

    attacker_to_bob = dh_public_key(prime, generator, attacker_private_a)
    attacker_to_alice = dh_public_key(prime, generator, attacker_private_b)

    alice_secret = dh_shared_secret(prime, attacker_to_alice, alice_private)
    bob_secret = dh_shared_secret(prime, attacker_to_bob, bob_private)
    attacker_secret_with_alice = dh_shared_secret(prime, alice_public, attacker_private_b)
    attacker_secret_with_bob = dh_shared_secret(prime, bob_public, attacker_private_a)

    message = b"hello bob"
    key_byte = alice_secret % 256
    encrypted = simple_xor_encrypt(message, key_byte)

    attacker_key_byte = attacker_secret_with_alice % 256
    attacker_decrypted = simple_xor_encrypt(encrypted, attacker_key_byte)

    bob_key_byte = bob_secret % 256
    bob_decrypted = simple_xor_encrypt(encrypted, bob_key_byte)

    print("Original message:", message)
```

```
    print("Encrypted bytes:", encrypted)
    print("Attacker reads message as:", attacker_decrypted)
    print("Bob reads message as:", bob_decrypted)

if __name__ == "__main__":
    demo_mitm_attack()
```

Code:

```
[Running] python -u "d:\Subjects\IS LAB\Lab 6\Task
Original message: b'hello bob'
Encrypted bytes: b'ehaab-obo'
Attacker reads message as: b'hello bob'
Bob reads message as: b'boffe*heh'

[Done] exited with code=0 in 0.274 seconds
```